以下是为您整理的关于 NES-VMC 算法在 NetKet 框架中复现的总结文档，包含目的、核心思想、技术演进与当前进展。

---

1 = NES-VMC 算法在 NETKET 官方 API 中的复现：目的与进展

1.1 = 1. 研究目的

目标：完全基于 NetKet 框架的高层 API（`MCState`、`VMC` 驱动）复现 **NES-VMC 算法**，用于计算量子多体系统（如 H_2 分子）的前 (K) 个激发态能量。

要求：

- 使用 NetKet 内置的扩展希尔伯特空间 `hi ** K`。
- 使用 `MCState` 管理变分态与采样器。
- 使用 `VMC` 驱动自动执行训练循环。
- 最终通过训练得到的模型，对角化平均局域能量矩阵，获得基态与激发态能量。

1.2 = 2. NES-VMC 算法核心思想

NES-VMC 将原系统前 (K) 个激发态的求解问题 **等价转化为一个“扩展系统”的基态求解问题**。

- **扩展希尔伯特空间**：由 (K) 个原系统副本张量积构成，每个配置对应 (K) 个组态 ((\mathbf{x}^1, \dots, \mathbf{x}^K))。
- **扩展系统波函数**：取为单粒子波函数的 Slater 行列式形式 [\Psi(\mathbf{x}^1, \dots, \mathbf{x}^K) = \det \begin{pmatrix} \psi_1(\mathbf{x}^1) & \cdots & \psi_K(\mathbf{x}^1) \ \vdots & \ddots & \vdots \ \psi_1(\mathbf{x}^K) & \cdots & \psi_K(\mathbf{x}^K) \end{pmatrix}]
- **局域能量**：不是通常的 ((H\Psi)/\Psi)，而是迹形式 [E_L(\mathbf{X}) = \operatorname{Tr}\bigl(M^{-1}(\mathbf{X}) , H_M(\mathbf{X}) \bigr)] 其中 (M) 为上述行列式矩阵，(H_M) 为哈密顿量作用在每一列上的矩阵。
- **能量提取**：优化结束后，对大量样本上的局域能量矩阵求平均，再对角化获得前 (K) 个本征能。

1.3 = 3. 技术实现演进与问题排查

1.3.1 = 3.1 初始方案：手动训练循环

实现方式：

- 手动调用 `sampler.sample()` 生成样本。
- 自定义损失函数（迹的平均值）与梯度计算。
- 使用 Optax 优化器手动更新参数。

优点：完全可控，逻辑清晰。 **缺点**：未利用 NetKet 官方高层 API，不符合用户要求。

1.3.2 = 3.2 尝试使用 VMC 驱动 + 自定义局域能量

理想接口：向 `nk.driver.VMC` 传入自定义的 `local_energy` 可调用对象。

实际结果：经过查阅源码与文档，**`nk.driver.VMC` 没有提供 `local_energy` 公开参数**。该方案不可行。

1.3.3 = 3.3 方案 1：继承 `MCState` 并重写 `local_estimators`

理论依据：

- `VMC` 驱动通过 `vstate.expect_and_grad(ham)` 计算能量与梯度。
- `expect_and_grad` 内部会调用 `vstate.local_estimators(ham)` 获得局域能量。
- 因此，只需继承 `MCState` 并重写 `local_estimators`，返回 NES-VMC 的迹即可。

遇到的问题：

- 在子类 `NESMCState` 的 `local_estimators` 方法中添加 `print` 语句，运行 `gs.run()` 后**无任何输出**。
- 表明 **`local_estimators` 可能未被调用**，或调用路径被 JIT 编译静默化。

尝试的调试手段：

- 改用 `jax.debug.print` 以在 JIT 编译函数内输出。
- 临时禁用 JIT（`jax.config.update('jax_disable_jit', True)`）观察 Python `print` 输出。
- 直接重写 `expect_and_grad` 以完全接管能量/梯度计算。

1.3.4 = 3.4 方案 1.1：重写 `expect_and_grad`（更彻底的自定义）

实现：

```python
class NESMCState(nk.vqs.MCState):
    def expect_and_grad(self, O, *, mutable=None, use_covariance=None):
        samples = self.samples
        model = self.model  # nnx 模型已包含最新参数
        local_energies = batch_local_energy(ha, model, samples)
        loss = local_energies.mean()
        # 手动计算梯度
        def loss_fn(vars): ...
        grad_vars = jax.grad(loss_fn)(nnx.state(model))
        return nk.stats.Stats(mean=loss, ...), (None, grad_vars)
```

遇到的问题：

- 在 `expect_and_grad` 中解包 `self.parameters` 时出现 `ValueError: too many values to unpack`。
- 原因：对于 `nnx` 模型，`MCState.parameters` 的 PyTree 结构在内部被 JIT 变换，不总是简单的 `(graphdef, variables)` 二元组。

修正：

- 直接使用 `self.model`（对 `nnx` 模块，`self.model` 自动携带最新参数）。
- 梯度计算时通过 `nnx.state(model)` 提取参数 PyTree。

1.4 = 4. 当前进展与代码状态

1.4.1 = 4.1 已完成部分

- ✅ 定义 H_2 分子系统，获取 FCI 基准能量。
- ✅ 构建扩展希尔伯特空间 `hi_ext = hi ** K` 及对应的 `TensorRule` 采样器。
- ✅ 定义 `NESModel`（NNX 模块），输出 (\log \det M)。
- ✅ 实现 `batch_local_energy` 函数，计算单个样本的迹。
- ✅ 创建 `NESMCState` 子类，重写 `local_estimators`（或 `expect_and_grad`），并加入调试输出。
- ✅ 使用 `nk.driver.VMC` 驱动训练。

1.4.2 = 4.2 待验证/解决的关键点

- **调用链验证**：确认自定义的 `local_estimators` 或 `expect_and_grad` 确实被 `VMC` 驱动调用。（建议临时禁用 JIT，观察控制台输出）
- **参数更新**：确保梯度计算正确并传递回 `MCState.parameters`，使模型参数更新。

- **激发态能量提取**：训练结束后，利用最终模型生成大量样本，计算平均局域能量矩阵并对角化。

1.4.3 = 4.3 推荐当前使用代码

```
# ...（前面的模型、哈密顿量定义不变）...

class NESMCState(nk.vqs.MCState):
    def local_estimators(self, op, *, chunk_size=None):
        jax.debug.print(">>> NESMCState.local_estimators called")
        samples = self.samples
        model = self.model
        local_energies = batch_local_energy(ha, model, samples)
        n_chains = self.sampler.n_chains
        chain_length = samples.shape[0] // n_chains
        return local_energies.reshape(n_chains, chain_length)

vstate = NESMCState(sampler, model, n_samples=1024, ...)
gs = nk.driver.VMC(ha, optimizer, variational_state=vstate)
gs.run(n_iter=300)
```

若此方法仍未显示调用，可采用**重写 `expect_and_grad`** 的备选方案（参考 3.4 节修正后的写法）。

1.5 = 5. 后续步骤

1. **调试调用链**：确保 `local_estimators` 或 `expect_and_grad` 被正确调用并输出损失值。
2. **完整训练**：运行 300 步左右，观察损失收敛。
3. **能量提取**：
   - 使用 `vstate.sample()` 获取最终样本。
   - 计算每个样本的局域能量矩阵并求平均。
   - 对角化获得本征值，与 FCI 基准对比。
4. **性能优化**：确认 JIT 编译正常，恢复 JIT 以获得训练速度。

文档更新日期：2026-04-17 **状态**：核心代码已完成，等待调试调用链与训练验证。